Energy Efficiency in Railway Scheduling

Paul Tennigkeit and Georgi Abshilava

University of Potsdam, Germany
{tennigkeit, abshilava}@uni-potsdam.de

1 Introduction

2 Encoding Energy Efficiency in ASP

2.1 What is Flatland?

Flatland is a framework for simulating railway networks. A Flatland environment consists of a two dimensional plane made up of different types of cells, some of which containing different types of interconnected tracks, and agents moving around on them from a starting point to an end point. Agents have an earliest departure and arrival time, and a pre-defined speed, represented by an inverse value. To limit the time of the simulation, a global time frame is set.

2.2 Baseline Encoding

The baseline ASP encoding (train_encoding.lp) handles all essential train and environment behavior, including defining legal transitions and movements, possible directions, and actions.

Directions There are four possible directions agents can go and point towards: North, south, east and west. Each is defined as a fact:

```
dir(n). dir(s). dir(e). dir(w).
```

Movements Each possible movement is defined as a fact:

```
moving(move_forward).
moving(move_left).
moving(move_right).
```

`delta()` defines what it means to "move" in each direction:

```
% The origin is located in the top left in Flatland, and
% the coordinates increment downwards

delta(A,s,1,0)  :- moving(A).
delta(A,n,-1,0) :- moving(A).
delta(A,e,0,1)  :- moving(A).
delta(A,w,0,-1) :- moving(A).
```

Transitions In order to represent different types of tracks, `transitions()` predicates are defined for each:

```
% ID  : track ID
% Dir1: pointing direction of agent
% Act : associated action with movement towards exit
% Dir2: exit direction
transition(ID, Dir1, Act, Dir2).
```

Due to the way this predicate is built, each track type is defined multiple times depending on its number of exit directions (e.g. straight tracks have two possible exits, north and south, and are, therefore, defined twice for each possible exit).

Actions The `action()` predicate is used as an interface between the Flatland visualization engine and the Clingo logic:

```
% train(ID): A train with an ID
% A        : An action
% T        : The time step action A is taken
action(train(ID),A,T) :- chosen(ID,T,A,_,_,_,_,_).
action(train(ID),move_forward,Dep) :- start(ID,_,Dep,_),
                                      Dep > 0.

action(train(ID),wait,T) :- start(ID,_,Dep,_),
                            Dep > 0,
                            T = 0..Dep-1.
```

Collision Constraints Like in real life, collisions between trains is to be avoided. To achieve this, the following constraints are employed:

```
% Trains can not occupy the same cell at the same time
```

```
:- stay(ID1,X,Y,T),
   stay(ID2,X,Y,T),
   ID1 < ID2.

% Trains can not swap their position
:- stay(ID1,X1,Y1,T),
   stay(ID2,X2,Y2,T),
   stay(ID1,X2,Y2,T+1),
   stay(ID2,X1,Y1,T+1),
   ID1 < ID2.
```

Reachability Constraints To simulate movement across the grid over time, a train's initial departure parameters are first converted into a `position()` state:

```
position(ID,(X,Y),Dep+1,Dir) :- start(ID,(X,Y),Dep,Dir).
```

From any current position, valid future states are derived using the `reachable()` predicate by checking cell track transitions, movement coordinate deltas, and the train's speed:

```
reachable(ID,T,A,X2,Y2,T2,Dir2) :- A != wait,
                                   position(ID,(X,Y),T,Dir1),
                                   cell((X,Y),Cell),
                                   transition(Cell,Dir1,A,Dir2),
                                   delta(A,Dir2,DX,DY),
                                   X2 = X + DX,
                                   Y2 = Y + DY,
                                   speed(ID,S),
                                   T2 = T + S,
                                   time(T2).

reachable(ID,T,wait,X,Y,T+1,Dir) :- position(ID,(X,Y),T,Dir),
                                    time(T+1).
```

At each discrete time step, a choice rule ensures exactly one valid transition is taken until the destination is reached:

```
1 { position(ID,(X2,Y2),T2,Dir2)
    : reachable(ID,T,A,X2,Y2,T2,Dir2) } 1
    :- position(ID,(_,_),T,_), not reached(ID,T).
```

In order to properly enforce the collision constraints, the time interval a train spends inside a specific cell must be tracked:

```
% define action predicate (this is just for visualization purposes)
chosen(ID,T,A,X,Y,X2,Y2,T2) :- position(ID,(X,Y),T,_),
                               position(ID,(X2,Y2),T2,_),
                               reachable(ID,T,A,X2,Y2,T2,_).
```

Because trains can have an inverse speed greater than 1 (meaning they traverse a cell over multiple time steps), the `stay()` predicate marks every time step a cell is occupied between entering and leaving:

```
% for inverse speed !=1: define interval to stay in a cell
stay(ID,X,Y,T1) :- chosen(ID,T,_,X,Y,_,_,T2),
                   time(T1), T <= T1, T1 < T2.
```

Target Arrival A train successfully completes its journey when its current coordinates match its designated target destination:

```
reached(ID,T) :- position(ID,(X,Y),T,_),
                 end(ID,(X,Y),_).
```

To ensure valid and bounded schedules, a look-ahead integrity constraint terminates search branches where a train can physically no longer reach its end coordinates within its individual allowable arrival deadline:

```
% stop searching if train is not in time
% to reach its target anymore
:- position(ID,(X,Y),T,_),
   end(ID,(EX,EY),Arr),
   speed(ID,InvSpd),
   T + (|X-EX| + |Y-EY|)*InvSpd > Arr.
```

By evaluating the remaining Manhattan distance multiplied by the train's inverse speed directly against its scheduled deadline, this single rule simultaneously prevents late arrivals and prunes unviable paths early in the search space.

Schedule Optimization Once valid routing pathways are established, the solver optimizes the schedule by minimizing overall transit delays. To ensure that trains are evaluated solely on their actual transit time rather than time spent idling at their destination after completion, the encoding isolates the precise time step T at which a train first reaches its end coordinates:

```
% the arrival times should be minimized
% (@2-higher priority than wait minimization)
reached_earlier(ID,T) :- reached(ID,T), reached(ID,T0), T0 < T.
arrival(ID,T) :- reached(ID,T), not reached_earlier(ID,T).
```

The `#minimize` statement aggregates the arrival times of all agents and assigns this objective a priority level of `@2`. This high priority ensures that in extended multi-objective encodings minimizing passenger schedule delay always takes precedence over secondary optimization metrics.

```
% minimize summed arrival times
#minimize { T@2, ID : arrival(ID,T) }.
```

2.3 Energy Encoding Mechanics

The energy efficiency encoding (train_encoding_energy.lp) adds new rules to represent the concept of energy and how different actions of the agents affect their usage and consumption of energy.

Cost Factors In real life, trains use a variety of motors that rely on different sources of energy, such as electric, hybrid, diesel or hydrogen power, which fundamentally affects how efficiently energy is consumed. The `pow_cost_fac()` predicate represents this baseline efficiency in the encoding by assigning an integer multiplier to each train based on its power type.

To then determine the true energy footprint of an agent, its mass must also be accounted for. This is handled by the `cost_fac()` rule, by multiplying a trains assigned cost factor, the `PCF`, by its defined weight `W`, resulting in a final, individualized cost factor `CF` for each train in the simulation:

```
% energy cost factor of trains based on power type + weight
```

```
pow_cost_fac(ID,1) :- pow_type(ID, electro).
pow_cost_fac(ID,2) :- pow_type(ID, hybrid).
pow_cost_fac(ID,3) :- pow_type(ID, diesel).
pow_cost_fac(ID,4) :- pow_type(ID, hydrogen).

cost_fac(ID,CF) :- pow_cost_fac(ID,PCF),
                   weight(ID,W),
                   CF = PCF * W.
```

Energy Costs of Actions Depending how fast the train moves or accelerates, it consumes different amounts of energy. These are reflected by the different `energy()` predicates:

```
% This rule calculates the energy cost of waiting,
% which is CF * 24 / S. The penalty is only applied
% once, when the train first hit the breaks.
% This behavior is reflected by the rule
% not prev_wait(ID,T).
energy(ID,T+1,E) :- action(train(ID),wait,T),
                    speed(ID,S),
                    not prev_wait(ID,T),
                    cost_fac(ID,CF),
                    E = CF * 24 / S.

% This rule calculates the energy cost of moving
% straight. This action's cost is only the base CF.
energy(ID,T2,E) :- action(train(ID),A,T),
                   A != wait,
                   position(ID,_,T,Dir),
                   speed(ID,S),
                   T2 = T + S,
                   position(ID,_,T2,Dir),
                   cost_fac(ID,CF),
                   E = CF .

% This rule calculates the energy cost of turning.
energy(ID,T2,E) :- position(ID,_,T,Dir1),
                   speed(ID,S),
```

```
                    T2 = T + S,
                    position(ID,_,T2,Dir2),
                    Dir1 != Dir2,
                    cost_fac(ID,CF),
                    E = CF * 12 / S.

% The prev_wait() rule is required in order to make sure,
% the agent is not constantly penalized for waiting in
% every time step.
prev_wait(ID,T) :- action(train(ID),_,T),
                   time(T-1),
                   action(train(ID),wait,T-1).
```

Optimizing the Energy Encoding The energy encoding adds the rule `#minimize { E@1,ID,T : energy(ID,T,E) }.` to also optimize fuel efficiency. As it is assigned the weight `@1`, the solver optimizes for arrival time first, which is assigned `@2`. Fuel efficiency is then optimized within the bounds of the best possible arrival times.

2.4 Results

3 Conclusion